

Questions probables et réponses

Préparation aux questions du jury, par thème

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	1 octobre 2026

Sommaire

Projet et périmètre

- Pouvez-vous résumer le projet en une minute ?
- Quelle est la part de data engineering et la part de data science ?
- Pourquoi ces cinq paires ?
- Pourquoi le profil day trading comme profil principal ?

Collecte (étape 1)

- Pourquoi deux sources, REST et WebSocket ?
- Comment respectez-vous les limites de Binance ?
- Pourquoi paginer sur la dernière bougie reçue plutôt qu'avec un compteur ?
- Pourquoi ne garder que les bougies marquées comme clôturées dans le flux WebSocket ?
- Comment gérez-vous les bougies manquantes ?

Bases de données (étape 2)

- Pourquoi deux bases de données ?
- PostgreSQL sait stocker du JSON. Pourquoi ne pas tout y mettre ?
- TimescaleDB est-il utile pour 2 millions de lignes ?
- Pourquoi une table par profil, et des vues ?
- Comment reliez-vous les deux bases ?
- Votre chargement est-il idempotent ?
- Pourquoi pas InfluxDB ou ClickHouse, conçus pour les séries temporelles ?

Machine learning (étape 3)

- Pourquoi une classification plutôt que prédire le prix ?
- Pourquoi une forêt aléatoire ?
- Pourquoi pas du deep learning, un LSTM par exemple ?
- 60 % de bonnes réponses, ce n'est pas peu ?
- Parlez-nous de la fuite de données que vous avez trouvée.
- Comment évitez-vous les fuites de données en général ?
- Pourquoi calibrer les probabilités ?
- Pourquoi deux seuils par style au lieu d'un seul ?
- À quoi sert l'extrait figé ?

API et sécurité (étape 4)

- Pourquoi des clés d'API et pas OAuth ou des jetons JWT ?
- Comment les clés sont-elles protégées ?
- Comment gérez-vous les secrets ?
- Pourquoi la clé ne va-t-elle jamais dans le navigateur ?
- Pourquoi n'y a-t-il pas de HTTPS ?

Conteneurisation

Pourquoi Docker Compose ?
Pourquoi des profils Compose ?
Pourquoi le modèle n'est-il pas dans l'image de l'API ?
Que deviennent les données si on supprime les conteneurs ?

Dérive des données

Comment mesurez-vous la dérive ?
Pourquoi le PSI plutôt qu'un test statistique comme Kolmogorov-Smirnov ?
La dérive est forte partout, mais le modèle garde ses performances. Comment l'expliquez-vous ?

Airflow et réentraînement

Pourquoi Airflow plutôt qu'une tâche cron ?
Pourquoi les DAG lancent-ils des commandes au lieu de contenir le code ?
Pourquoi le LocalExecutor et pas Celery ?
Comment fonctionne le duel champion contre challenger ?
Pourquoi le challenger n'a-t-il jamais gagné ?

CI/CD

Que vérifie votre intégration continue ?
Combien de tests avez-vous, et quelle est la couverture ?
Pourquoi des doublures plutôt que de vraies bases dans les tests ?
Pourquoi Watchtower plutôt qu'un déploiement poussé par GitHub ?
Avez-vous rencontré un problème avec Watchtower ?

Supervision

Pourquoi Prometheus et Grafana ?
Quelles questions vos alertes surveillent-elles ?
Pourquoi un exportateur à part ?
Les alertes préviennent-elles quelqu'un ?

Rentabilité et trading

Votre modèle n'est pas rentable. À quoi sert le projet ?
Pourquoi avoir raison 60 % du temps ne suffit-il pas ?
Un horizon plus long ne résoudrait-il pas le problème des frais ?
Votre bot vend à découvert. Est-ce possible sur le marché spot ?
Mettriez-vous de l'argent réel derrière ce bot ?

Limites et suites

Que feriez-vous avec plus de temps ?
Quelle est la principale limite technique ?
Si c'était à refaire, que changeriez-vous ?

Questions pièges et culture générale

ETL ou ELT : où se situe votre pipeline ?
Votre pipeline est-il batch ou streaming ?
Votre base est-elle OLTP ou OLAP ?

Pourquoi pas Kafka ?

Pourquoi pas Kubernetes ?

Pourquoi pas le cloud ?

Que se passerait-il avec 500 paires au lieu de 5 ?

Combien coûte votre infrastructure ?

Le RGPD vous concerne-t-il ?

Que se passe-t-il si Binance change son API ?

Que se passe-t-il si la base tombe ?

Que se passe-t-il si deux collectes se chevauchent ?

Si on ne sait pas

Projet et périmètre

Pouvez-vous résumer le projet en une minute ?

Nous collectons les bougies de 5 paires Binance, en historique par l'API REST et en direct par WebSocket. Les réponses brutes vont dans MongoDB, les bougies nettoyées dans PostgreSQL avec TimescaleDB. Un modèle prédit le sens de la prochaine bougie et répond acheter, vendre ou attendre selon un style conservateur ou agressif. Il est servi par une API FastAPI protégée par clé, et Airflow fait tourner la collecte, la mesure de dérive et le réentraînement sans intervention, avec une CI GitHub Actions, un déploiement par Watchtower et une supervision Prometheus et Grafana.

Quelle est la part de data engineering et la part de data science ?

La plus grande part du projet est du data engineering : collecte paginée et respectueuse des quotas, deux couches de stockage, chargement idempotent, API, conteneurs, orchestration, CI/CD et supervision. Le machine learning occupe l'étape 3, et nous l'avons traité avec une démarche d'ingénieur : extrait figé et signé, découpage chronologique, mesure sur des données jamais vues. Même le réentraînement est surtout un problème d'ingénierie : reproductibilité, publication atomique du fichier, registre des modèles.

Pourquoi ces cinq paires ?

BTC, ETH, BNB, SOL et XRP contre USDT font partie des paires les plus échangées. Un marché très actif donne des bougies régulières, sans trous ni prix aberrants dus à l'absence d'échanges. Cinq paires suffisent pour vérifier qu'un modèle se généralise d'un actif à l'autre, sans multiplier le temps de collecte.

Pourquoi le profil day trading comme profil principal ?

Ses pas de temps (15 minutes, 1 heure, 4 heures) offrent un bon compromis : deux ans d'historique couvrent plusieurs régimes de marché, et le 15 minutes fournit beaucoup d'exemples pour apprendre. Le scalping n'a que 180 jours d'historique, et le swing produit trop peu de bougies pour entraîner un modèle. Nous avons concentré l'effort de modélisation sur un seul profil plutôt que d'en mener trois à moitié.

Collecte (étape 1)

Pourquoi deux sources, REST et WebSocket ?

Elles répondent à deux questions différentes. Le REST répond à « que s'est-il passé ? » et sert à constituer l'historique ; le WebSocket répond à « que se passe-t-il maintenant ? » et pousse les

bougies en continu. Les deux sont converties vers le même schéma pivot de 13 colonnes, donc la suite du projet ignore d'où vient une bougie.

Comment respectez-vous les limites de Binance ?

Binance renvoie au plus 1 000 bougies par appel et accorde 6 000 unités de poids par minute et par adresse IP. Le client lit le poids consommé dans l'en-tête de chaque réponse, se met en pause avant la limite et respecte le délai `Retry-After` en cas d'erreur 429, car insister mène à un bannissement temporaire. Un appel coûte le même poids pour 1 ou 1 000 bougies, donc nous demandons toujours le maximum. Si le serveur principal ne répond pas, le client bascule sur le miroir public `data-api.binance.vision`.

Pourquoi paginer sur la dernière bougie reçue plutôt qu'avec un compteur ?

Un compteur suppose que chaque lot est complet. Si Binance renvoie moins de 1 000 bougies, par exemple après un arrêt de la plateforme, le compteur saute des données sans le signaler. En repartant de l'heure d'ouverture de la dernière bougie reçue plus une milliseconde, la pagination reste juste quel que soit le contenu des lots.

Pourquoi ne garder que les bougies marquées comme clôturées dans le flux WebSocket ?

Le WebSocket renvoie la bougie en cours à chaque transaction, et seule la version envoyée à la clôture est définitive. Sur une mesure de 200 secondes, nous avons reçu 500 messages pour 20 bougies : sans ce filtre, 96 % des lignes seraient des doublons à des prix jamais confirmés. La même règle vaut pour le modèle : il ne prédit jamais à partir d'une bougie en cours.

Comment gérez-vous les bougies manquantes ?

Nous les mesurons et nous ne les comblons pas. Binance a connu une dizaine d'arrêts entre 2020 et 2023 : en pas horaire, ils donnent 99,96 % de complétude ; en 4 heures, ils disparaissent dans une bougie au volume anormalement faible. Remplir ces trous inventerait des prix qui n'ont jamais existé, puisque le marché était fermé. En production, le contrôle qualité vérifie la complétude sur deux jours à chaque passage.

Bases de données (étape 2)

Pourquoi deux bases de données ?

Nous répartissons selon la forme de la donnée. Une bougie a toujours les mêmes 13 colonnes et une clé métier sans doublon : c'est du relationnel, dans PostgreSQL. Les réponses brutes de Binance et les métadonnées comme `exchangeInfo` ont des structures variables et imbriquées : ce sont des documents, dans MongoDB. Cela donne deux couches, une couche brute rejouable et une couche exploitable.

PostgreSQL sait stocker du JSON. Pourquoi ne pas tout y mettre ?

Ce serait faisable, et nous utilisons d'ailleurs le JSONB de PostgreSQL pour les variables techniques. Nous avons gardé MongoDB pour la couche brute parce qu'elle a un rôle différent : archiver l'original sans transformation, et le relire si le nettoyage change. MongoDB nous apporte aussi deux mécanismes utiles ici : un index TTL qui efface le carnet d'ordres après 30 jours sans traitement de purge, et un index partiel limité aux bougies clôturées du flux. Le prix est une base de plus à opérer, que nous avons jugé acceptable dans Docker.

TimescaleDB est-il utile pour 2 millions de lignes ?

Pour la performance brute, pas encore : PostgreSQL commence à ralentir autour de quelques dizaines de millions de lignes. Nous l'avons adopté parce qu'il ne coûtait rien, l'image Docker officielle fournissant PostgreSQL avec l'extension. Il apporte le découpage en tranches de temps, la compression des tranches anciennes et des politiques par table, donc par profil, qui serviront quand le volume grandira.

Pourquoi une table par profil, et des vues ?

Chaque pas de temps est stocké une seule fois, dans la table du profil qui le garde le plus longtemps : le 15 minutes chez le day trading, le 4 heures chez le swing. Les vues `v_scalping`, `v_day_trading` et `v_swing` rendent à chaque profil ses trois pas de temps, et l'API lit toujours une vue. La règle est inscrite dans la table `profile_intervals` avec le champ `is_owner`, donc changer la répartition ne demande pas de toucher au code. Le découpage permet aussi des tranches et une compression différentes par profil.

Comment reliez-vous les deux bases ?

Aucun moteur ne peut imposer une clé étrangère vers l'autre. Chaque bougie de PostgreSQL porte un champ `raw_ref` qui désigne le document MongoDB d'où elle vient, et la clé métier est présente des deux côtés. Depuis n'importe quelle ligne, on retrouve la réponse exacte de Binance qui l'a produite.

Votre chargement est-il idempotent ?

Oui. L'écriture se fait par `INSERT ... ON CONFLICT DO UPDATE` sur la clé métier : relancer un chargement met à jour les bougies existantes sans jamais les dupliquer. C'est ce qui permet à Airflow de relancer une tâche après un échec sans risque. Nous avons aussi découvert que le pilote ne comptait que le dernier lot d'une écriture groupée, 124 lignes annoncées pour 70 124 écrites : le journal des chargements interroge donc la table.

Pourquoi pas InfluxDB ou ClickHouse, conçus pour les séries temporelles ?

Ils sont dimensionnés pour des milliards de lignes et n'offrent pas les garanties transactionnelles de PostgreSQL. Avec 2 millions de bougies, nous n'avons pas besoin de leur vitesse d'ingestion, et nous voulions les contraintes d'intégrité, les jointures avec les référentiels et les vues. TimescaleDB nous donne les fonctions temporelles en restant dans PostgreSQL.

Machine learning (étape 3)

Pourquoi une classification plutôt que prédire le prix ?

Prédire le prix donne un score trompeur. Un modèle naïf qui répond « le prochain prix sera le prix actuel » obtient un R^2 de 0,999998, et les vrais modèles font moins bien que lui. Ce R^2 mesure surtout que le bitcoin vaut à peu près le même prix qu'une heure plus tôt. Le sens de la bougie suivante est la question utile pour décider, et elle se mesure honnêtement.

Pourquoi une forêt aléatoire ?

Elle a eu le meilleur score en validation face au gradient boosting et à la régression logistique. Elle supporte bien des variables corrélées et bruitées, demande peu de réglages et donne des probabilités que nous calibrons ensuite. Nous l'avons réglée avec 400 arbres et au moins 100 exemples par feuille, pour limiter le surapprentissage sur un signal faible.

Pourquoi pas du deep learning, un LSTM par exemple ?

Nous ne l'avons pas fait. Notre diagnostic est que la limite vient du signal lui-même : même en sachant à l'avance quelles bougies bougeront le plus, le sens n'y est prédit qu'à 48 %, et toutes nos pistes de variables ont plafonné au même niveau. Un LSTM aurait demandé beaucoup plus de temps d'entraînement et de réglage pour un gain incertain, et il est plus difficile à expliquer et à reproduire. Si nous le testions, ce serait avec exactement le même protocole : extrait figé, découpage chronologique, mesure après frais.

60 % de bonnes réponses, ce n'est pas peu ?

Sur une question aussi difficile que le sens d'une bougie de 15 minutes, le hasard est à 50 %, et notre première approche n'en tirait que 50,6 %. Atteindre 60 % sur les bougies où le modèle se prononce était l'objectif fixé, et l'intervalle de confiance à 95 % va de 55,4 % à 64,3 % : la borne basse reste nettement au-dessus du hasard. Le problème est ailleurs : avec des mouvements de 0,227 % en moyenne et 0,2 % de frais, il faudrait 94 % de bonnes réponses pour gagner de l'argent.

Parlez-nous de la fuite de données que vous avez trouvée.

En ajoutant le contexte des bougies 1 heure et 4 heures, une bougie 15 minutes de 12 h 15 recevait les indicateurs de la bougie 4 heures ouverte à midi, qui ne se termine qu'à 16 h : quatre heures de futur. Le score est passé de 0,52 à 0,61, et c'est justement ce saut trop beau qui nous a alertés. En ne joignant que les bougies lentes déjà clôturées, il est retombé à 0,52. Un test automatisé empêche désormais cette erreur de revenir.

Comment évitez-vous les fuites de données en général ?

Par trois règles. Le découpage est chronologique, avec `TimeSeriesSplit` pour la validation croisée : on apprend toujours sur le passé et on mesure sur le futur. Les variables ne regardent que des bougies clôturées, et un test vérifie qu'une variable a la même valeur quelle que soit la longueur

d'historique utilisée. La mesure finale porte sur 14 831 bougies postérieures à l'extrait, sur lesquelles aucune décision n'a été prise.

Pourquoi calibrer les probabilités ?

Une forêt aléatoire non calibrée est trop prudente sur les probabilités extrêmes, et ce sont précisément celles où un modèle sélectif décide. La calibration isotonique, faite sur une période que le modèle n'a pas vue, fait que 58 % annoncés correspondent à environ 58 hausses sur 100. C'est la seule piste qui a apporté un gain confirmé : 1,5 à 1,7 point d'accuracy sélective.

Pourquoi deux seuils par style au lieu d'un seul ?

Les probabilités du modèle ne sont pas symétriques : elles descendent bas du côté de la baisse mais ne dépassent jamais 0,564 du côté de la hausse. Avec un seuil unique mesuré depuis 0,5, le style conservateur se trouvait au-dessus de ce maximum et ne pouvait jamais acheter. Chaque seuil est donc réglé pour que chaque côté déclenche la même part d'ordres.

À quoi sert l'extrait figé ?

Les données en base changent tous les jours. Entraîner deux fois sur « la base » donne deux modèles différents sans que le code ait changé. L'extrait est une copie Parquet coupée à une date, avec une empreinte SHA-256 vérifiée avant chaque entraînement et enregistrée dans MLflow. Relancé trois fois le même jour, le duel de réentraînement a donné les mêmes chiffres à la quatrième décimale.

API et sécurité (étape 4)

Pourquoi des clés d'API et pas OAuth ou des jetons JWT ?

Les clients de notre API sont des programmes, l'interface et Airflow, sans utilisateur humain qui se connecte avec un mot de passe. OAuth et JWT répondent au cas d'utilisateurs qui s'authentifient et reçoivent un jeton temporaire, ou d'une application qui agit pour le compte d'un tiers. Pour du machine à machine, la clé d'API est le schéma standard, celui de Binance, Stripe ou GitHub. Si l'API devait servir des personnes avec des droits différents, nous passerions à OAuth 2 avec des jetons à durée limitée.

Comment les clés sont-elles protégées ?

Chaque client a sa propre clé de 64 caractères hexadécimaux, générée par le module `secrets`, ce qui permet d'en révoquer une sans couper l'autre. La comparaison se fait avec `hmac.compare_digest`, en temps constant, pour que le temps de réponse ne révèle rien. Si aucune clé n'est configurée, l'API répond 503 : un oubli de configuration ne l'ouvre jamais. Un test parcourt toutes les routes et échoue si l'une d'elles accepte un appel sans clé.

Comment gérez-vous les secrets ?

Ils sont dans un fichier `.env`, créé par `python -m scripts.generer_secrets` avec des valeurs aléatoires, et jamais versionné. Dans la CI, le déploiement complet est testé avec des secrets neufs générés à la volée. La limite est que ce fichier n'est protégé que par ses droits sur le disque : en production, nous utiliserions un coffre à secrets.

Pourquoi la clé ne va-t-elle jamais dans le navigateur ?

Tout ce qui est envoyé à un navigateur peut être lu par n'importe qui. La page appelle `/api/...`, et c'est nginx, côté serveur, qui transmet la requête à l'API en y ajoutant la clé de l'interface. L'API elle-même n'a aucun port publié : seuls nginx, Airflow et Prometheus la joignent dans le réseau Docker.

Pourquoi n'y a-t-il pas de HTTPS ?

Tous les ports publiés sont liés à `127.0.0.1` : rien n'est joignable depuis le réseau, donc rien ne circule en clair hors de la machine. Avant toute ouverture, nous ajouterions une terminaison TLS devant nginx. Nous ne l'avons pas fait parce que l'application tourne aujourd'hui sur une seule machine locale.

Conteneurisation

Pourquoi Docker Compose ?

Il décrit les 16 services, leurs volumes, leurs réseaux et leurs dépendances dans un seul fichier, et les démarre en une commande. La même image donne le même comportement sur toutes les machines, ce que la CI vérifie en démarrant toute l'application sur une machine vierge. Pour une seule machine, c'était l'outil adapté ; Kubernetes ne se justifierait qu'avec plusieurs serveurs.

Pourquoi des profils Compose ?

Le produit, c'est-à-dire les bases, l'API et l'interface, démarre toujours. Les profils `pipeline`, `supervision`, `outils` et `deploiement` s'ajoutent à la demande. Un développeur qui travaille sur l'API n'a pas besoin de lancer Airflow, Prometheus et Grafana.

Pourquoi le modèle n'est-il pas dans l'image de l'API ?

Il est monté depuis `./models` en lecture seule. Un nouveau modèle se met ainsi en service sans reconstruire ni redéployer l'image, ce qui permet au réentraînement hebdomadaire de publier seul. L'image reste légère : ses dépendances sont listées à part, sans MLflow, Jupyter ni matplotlib.

Que deviennent les données si on supprime les conteneurs ?

Elles sont conservées dans des volumes Docker, en dehors des conteneurs. Recréer le conteneur d'une base ne perd rien. En revanche, nous n'avons pas de sauvegarde de ces volumes vers une

autre machine : c'est une des suites prévues.

Dérive des données

Comment mesurez-vous la dérive ?

Avec l'indice PSI, par paire et par pas de temps, sur une fenêtre de 90 jours. Pour chaque variable, on découpe la répartition d'apprentissage en déciles, puis on regarde comment les valeurs récentes se répartissent dans ces mêmes tranches. Sous 0,10 c'est stable, entre 0,10 et 0,25 c'est une dérive modérée, au-delà une dérive forte.

Pourquoi le PSI plutôt qu'un test statistique comme Kolmogorov-Smirnov ?

Le PSI se lit directement avec des seuils d'usage reconnus, et il se calcule sur des déciles, donc il reste rapide et compréhensible. Un test comme Kolmogorov-Smirnov devient significatif pour de très petits écarts dès que l'échantillon est grand, ce qui est notre cas avec des milliers de bougies. Nous ne l'avons pas mis en place en parallèle.

La dérive est forte partout, mais le modèle garde ses performances. Comment l'expliquez-vous ?

Les 15 séries surveillées étaient en dérive forte, et le modèle en service gardait pourtant 57,5 % de bonnes réponses, dans la marge des 60 % mesurés à l'étape 3. La dérive porte surtout sur la volatilité : le marché bouge avec une amplitude différente, mais la relation entre nos variables et le sens de la bougie suivante semble tenir. Une dérive des données n'implique pas une dérive du concept. C'est pour cela que la dérive déclenche une comparaison, et non une mise en service automatique.

Airflow et réentraînement

Pourquoi Airflow plutôt qu'une tâche cron ?

Cron lance une commande à heure fixe, et c'est tout. Airflow ajoute l'ordre entre les tâches, les nouvelles tentatives espacées, l'interdiction de deux exécutions simultanées et l'historique de chaque exécution avec ses journaux. Quand une collecte échoue, on voit laquelle, quand et pourquoi.

Pourquoi les DAG lancent-ils des commandes au lieu de contenir le code ?

Airflow orchestre, il ne calcule pas. Chaque tâche lance la commande que l'on taperait à la main, et la logique reste dans les scripts, où elle est testée. Une panne se reproduit en recopiant une ligne de commande. Les scripts tournent dans un environnement Python séparé de celui d'Airflow, pour éviter les conflits de versions et entraîner avec la version exacte de scikit-learn que l'API utilise.

Pourquoi le LocalExecutor et pas Celery ?

Nous avons quelques tâches toutes les 15 minutes, sur une seule machine. Celery ajouterait une file de messages et des machines de calcul sans rien apporter à ce volume. Le passage à un exécuteur distribué se ferait si les tâches ne tenaient plus sur une machine.

Comment fonctionne le duel champion contre challenger ?

Le modèle en service et le nouveau sont mesurés sur les 21 derniers jours, retirés de l'apprentissage, avec un jour d'écart parce que l'étiquette d'une bougie dépend de la suivante. Le challenger ne remplace le champion que s'il a au moins 30 ordres et une accuracy au moins égale en style conservateur. La publication copie le fichier puis le renomme, et l'ancien est archivé pour pouvoir revenir en arrière.

Pourquoi le challenger n'a-t-il jamais gagné ?

Sur les deux duels réels, le champion a eu 0,575 contre 0,558, puis 0,594 contre 0,571. Les écarts sont dans les marges d'erreur : le 23 septembre, 0,529 à 0,620 contre 0,516 à 0,600. Les deux modèles se valent, et à valeur égale ou incertaine, la règle garde celui qui est déjà en service. C'est le comportement voulu : remplacer un modèle a un coût et un risque, et il faut une preuve pour le faire.

CI/CD

Que vérifie votre intégration continue ?

Cinq contrôles tournent en parallèle à chaque envoi sur la branche principale : analyse statique avec ruff, les 149 tests avec la couverture, le chargement des DAG dans Airflow, la validité des règles Prometheus et de la configuration nginx, et un déploiement complet de zéro sur une machine vierge vérifié de l'extérieur. Les images ne sont publiées que si les cinq réussissent. L'ensemble dure environ 5 minutes.

Combien de tests avez-vous, et quelle est la couverture ?

149 tests pytest, qui passent en une quinzaine de secondes, plus 21 vérifications de l'application démarrée. La couverture se mesure dans la CI : elle est élevée sur ce qui tourne en production, autour de 82 % pour les routes de l'API, 98 % pour la sécurité et 97 % pour le calcul des variables. Rapportée à tout le dépôt, elle tombe à 21 %, parce que les scripts d'analyse de l'étape 3 ne sont pas testés : ce sont des expériences exécutées une fois, dont les résultats sont enregistrés. Le carnet de positions est la partie de production la moins couverte, ce serait la priorité.

Pourquoi des doublures plutôt que de vraies bases dans les tests ?

Pour que les tests passent sur n'importe quelle machine sans Docker, en quelques secondes, et qu'un échec désigne un bug du code et non une base éteinte. Par exemple, un faux modèle réglé sur 0,58 permet de vérifier que le style agressif achète et que le conservateur s'abstient. Ce que les

doublures ne peuvent pas voir, comme la clé ajoutée par nginx ou la limitation de débit, est vérifié par le script qui interroge l'application démarrée.

Pourquoi Watchtower plutôt qu'un déploiement poussé par GitHub ?

Watchtower fonctionne en mode pull : la machine va chercher les nouvelles images toutes les 5 minutes, rien n'est ouvert vers l'extérieur et GitHub n'exécute jamais de code chez nous. L'alternative, un exécuteur GitHub installé sur la machine, a été écartée parce que le dépôt est public : du code proposé depuis un fork aurait pu y être exécuté. Watchtower ne touche que les conteneurs marqués d'une étiquette, jamais les bases de données.

Avez-vous rencontré un problème avec Watchtower ?

Oui, au premier déploiement. Watchtower a remplacé les six conteneurs construits localement par les images de la CI, qui n'ont pas les étiquettes que Docker Compose utilise pour retrouver ses conteneurs. Compose ne les reconnaissait plus, et nous les avons recréés. La leçon : sur une machine de production, on ne construit pas d'image localement, on part de celles de la CI.

Supervision

Pourquoi Prometheus et Grafana ?

Ce sont les outils libres de référence pour les métriques et les tableaux de bord, et ils tournent dans Docker comme le reste. Prometheus interroge chaque service toutes les 15 secondes et évalue 12 règles d'alerte ; un service arrêté se voit immédiatement puisqu'il ne répond plus. Le tableau Grafana est décrit dans un fichier JSON versionné, donc identique sur toute machine.

Quelles questions vos alertes surveillent-elles ?

Le service répond-il, les données arrivent-elles, le bot tourne-t-il, le modèle est-il encore adapté, et quelqu'un force-t-il l'accès. Chaque alerte doit durer avant de se déclencher, de 2 minutes à 1 heure selon le cas, pour qu'un redémarrage de quelques secondes n'en produise pas. Les règles sont versionnées et vérifiées par la CI.

Pourquoi un exportateur à part ?

Les questions les plus importantes ne concernent pas l'API : les données arrivent-elles, le dernier réentraînement a-t-il eu lieu ? Les réponses sont en base, écrites par Airflow. L'exportateur les lit et les traduit en métriques, il tourne à part pour survivre à une panne de l'API, et garde ses lectures en cache une minute pour ne pas charger la base.

Les alertes préviennent-elles quelqu'un ?

Non. Elles se consultent dans Grafana et Prometheus, personne ne reçoit de message. La suite naturelle est Alertmanager, vers un courriel ou une messagerie. Nous ne l'avons pas fait par

manque de temps, et parce que sans personne d'astreinte, une notification aurait surtout servi la démonstration.

Rentabilité et trading

Votre modèle n'est pas rentable. À quoi sert le projet ?

Le projet demandé est une chaîne de données complète, et elle fonctionne de bout en bout : collecte, stockage, modèle mesuré, API, automatisation, supervision. Le modèle atteint l'objectif fixé de 60 %, et notre chaîne nous permet de dire précisément pourquoi il ne gagne pas d'argent. Une chaîne qui aurait annoncé un bot rentable sans backtest après frais aurait été plus inquiétante qu'utile.

Pourquoi avoir raison 60 % du temps ne suffit-il pas ?

Sur une bougie de 15 minutes, le prix bouge en moyenne de 0,227 %, et un aller-retour coûte 0,2 % de frais. Pour gagner en moyenne, il faudrait avoir raison 94 % du temps. Au backtest sur les bougies jamais vues, le style conservateur perd 3,6 % et l'agressif 5,0 %, alors que sans frais ils gagneraient 1,0 et 0,8 %.

Un horizon plus long ne résoudrait-il pas le problème des frais ?

Nous l'avons testé. En 4 heures, le mouvement moyen est de 0,67 % et il faudrait 65 % de bonnes réponses ; nous obtenons 53 à 57 %. En 1 jour, il faudrait 56 % et nous obtenons 48 à 58 %, de façon instable d'une période à l'autre. Sur le rejeu d'un an, le 4 heures termine pourtant légèrement positif, à 10 154 euros pour 10 000 investis, mais ce n'est pas assez stable pour en tirer une stratégie.

Votre bot vend à découvert. Est-ce possible sur le marché spot ?

Non, et c'est une limite académique de notre simulation. Sur le spot, on ne peut vendre que ce que l'on possède : seuls les ordres à la hausse sont possibles. Parier à la baisse demande les contrats à terme ou la marge, avec d'autres frais et d'autres risques, comme les frais de financement et la liquidation, que nous ne simulons pas. Nos backtests appliquent le tarif spot de 0,1 % par ordre ; au tarif des contrats à terme, moins élevé, le rejeu en 15 minutes se rapproche de l'équilibre, mais sans compter le financement ni le glissement de prix.

Mettriez-vous de l'argent réel derrière ce bot ?

Non. Le backtest après frais est négatif, et le ratio d'excursion médian de 0,92 montre que le prix va en moyenne un peu plus loin contre la position que pour elle. L'avantage du modèle sur le sens d'une bougie ne se prolonge pas au-delà. C'est pour cela que chaque réponse de l'API porte un avertissement et que le carnet est virtuel.

Limites et suites

Que feriez-vous avec plus de temps ?

D'abord changer de cible : ne prédire que les mouvements assez grands pour couvrir les frais, plutôt que le sens de chaque bougie. Côté ingénierie, ajouter les notifications d'alerte avec Alertmanager, une sauvegarde des volumes et un hébergement sur un serveur. Enfin, une terminaison HTTPS et un coffre à secrets avant toute ouverture au réseau.

Quelle est la principale limite technique ?

Tout tourne sur une seule machine : si elle s'arrête, tout s'arrête. Le carnet de positions rattrape les bougies manquées au redémarrage et l'ingestion comble son retard en un passage, donc on ne perd pas de données de marché, mais il n'y a ni redondance ni sauvegarde externe.

Si c'était à refaire, que changeriez-vous ?

Nous mesurerions plus tôt l'écart entre le mouvement moyen et les frais. Ce calcul simple aurait montré dès le début que le sens d'une bougie de 15 minutes ne pouvait pas être rentable, et nous aurions orienté la cible plus vite. Nous aurions aussi écrit le test anti-fuite avant d'ajouter le contexte multi-échelles, plutôt qu'après.

Questions pièges et culture générale

ETL ou ELT : où se situe votre pipeline ?

Plutôt ELT pour la partie brute : nous chargeons la réponse de Binance telle quelle dans MongoDB, puis nous la transformons. Mais la transformation se fait en Python avant l'écriture dans PostgreSQL, ce qui relève de l'ETL pour la couche exploitable. L'intérêt de garder le brut est de pouvoir rejouer un nettoyage corrigé sans redemander les données à Binance.

Votre pipeline est-il batch ou streaming ?

Les deux existent, mais la production est en micro-batch. Le collecteur WebSocket sait suivre le marché en continu, mais l'ingestion automatisée passe toutes les 15 minutes, ce qui correspond au pas de temps principal du modèle. Pour des décisions sur des bougies de 15 minutes, un traitement en flux n'apporterait rien et ajouterait de la complexité.

Votre base est-elle OLTP ou OLAP ?

PostgreSQL est un moteur transactionnel, et notre usage est mixte : beaucoup d'écritures en petits lots toutes les 15 minutes, et des lectures analytiques par période pour le modèle et Grafana. TimescaleDB rapproche le moteur d'un usage analytique avec la compression et le découpage par temps. À notre volume, un entrepôt OLAP dédié n'était pas nécessaire.

Pourquoi pas Kafka ?

Kafka sert à transporter de gros flux d'événements entre de nombreux producteurs et consommateurs, avec rejeu. Nous avons une source, Binance, et un débit de quelques centaines de bougies toutes les 15 minutes. La base sert déjà de point de reprise : l'ingestion repart de la dernière bougie stockée. Kafka deviendrait pertinent avec un vrai traitement en flux, beaucoup de paires et plusieurs consommateurs du même flux.

Pourquoi pas Kubernetes ?

Kubernetes répartit des conteneurs sur plusieurs machines et gère la haute disponibilité. Nous avons une machine et 16 services : Docker Compose les décrit en un fichier et la CI les démarre de zéro en quelques minutes. Kubernetes ajouterait une couche d'exploitation importante sans bénéfice à cette échelle. Le passage à plusieurs serveurs serait le bon moment pour y aller.

Pourquoi pas le cloud ?

Nous n'avons pas déployé sur un fournisseur cloud. Les services managés comme Snowflake ou BigQuery sont payants et hébergés, alors que la consigne demandait une application conteneurisée. Comme tout est en conteneurs, le passage sur une machine virtuelle cloud serait direct : même fichier Compose, mêmes images publiées sur ghcr.io, avec HTTPS et un coffre à secrets à ajouter.

Que se passerait-il avec 500 paires au lieu de 5 ?

La collecte est générique, ajouter une paire ne demande pas de code, mais le quota de Binance deviendrait la contrainte : il faudrait répartir les appels dans le temps ou passer au WebSocket, qui permet de s'abonner à de nombreux flux. Côté base, on passerait à des centaines de millions de lignes, là où TimescaleDB et sa compression prennent leur sens. Le tour du bot, qui prend environ 90 secondes pour 30 combinaisons, devrait être parallélisé, et c'est là qu'un exécutateur distribué comme Celery deviendrait utile. Nous ne l'avons pas mesuré.

Combien coûte votre infrastructure ?

Rien en licences : tous les composants sont libres et tournent sur une seule machine. GitHub Actions est gratuit pour un dépôt public comme le nôtre. Les frais réels seraient ceux d'un serveur pour un hébergement permanent, que nous n'avons pas chiffré.

Le RGPD vous concerne-t-il ?

Très peu. Nous ne traitons que des données de marché publiques, sans aucune donnée personnelle. Les seules informations liées à des personnes seraient les adresses IP dans les journaux de nginx. Si l'API servait des utilisateurs, il faudrait définir une durée de conservation de ces journaux et des comptes.

Que se passe-t-il si Binance change son API ?

La conversion vers le schéma pivot est écrite à un seul endroit, donc la correction serait localisée. Si le format change, les contraintes de la base refusent les données incohérentes, le contrôle qualité échoue et l'alerte « plus de bougie depuis 3 intervalles » se déclenche. Comme les réponses brutes sont archivées dans MongoDB, on pourrait aussi analyser le nouveau format sur des données réelles. En revanche, nos tests utilisent des doublures et ne détecteraient pas seuls un changement de Binance.

Que se passe-t-il si la base tombe ?

L'API répond 503 sur `/health` au lieu de planter, et Docker relance automatiquement un conteneur qui s'arrête (politique `restart: unless-stopped`) ; un conteneur qui tourne mais ne répond plus est signalé par sa sonde de santé, sans être relancé tout seul. Les tâches Airflow font jusqu'à trois essais, avec un délai qui double à chaque fois. Prometheus déclenche une alerte critique sur la base illisible et le retard des données. Une fois la base revenue, l'ingestion rattrape tout en un passage et le carnet rejoue les bougies manquées. Les données sont dans des volumes, donc rien n'est perdu tant que le disque de la machine n'est pas perdu.

Que se passe-t-il si deux collectes se chevauchent ?

Airflow l'empêche : chaque DAG est limité à une exécution à la fois. Et même si cela arrivait, par exemple avec un lancement manuel, l'écriture idempotente sur la clé métier ferait que la même bougie serait mise à jour deux fois, jamais dupliquée.

Si on ne sait pas

« Nous ne l'avons pas mesuré dans le projet, donc je ne veux pas vous donner un chiffre au hasard ; voici comment nous le mesurerions. »

« Je ne connais pas bien cet outil. Ce que je peux vous dire, c'est le besoin auquel nous avons répondu et pourquoi notre choix suffisait à notre échelle. »

« C'est une bonne question, que nous ne nous étions pas posée sous cet angle. Ma première réponse serait la suivante, sous réserve de vérification. »

« Cette partie a été principalement réalisée par mon binôme, qui pourra compléter ; voici ce que j'en ai compris. »